



Empresa Operativa Chile

Crear, operar y controlar una empresa chilena a través del tiempo

Contabilidad y tributación explicables · Reglas versionadas por año · Evidencia obligatoria
Sin servidor · Sin cuentas · Sin telemetría · Los datos no salen del dispositivo

Android · APK

Windows · MSI/EXE

Navegador · PWA

CLI · Node 20+

● EMPRESA REAL

Tu contabilidad, con bitácora de auditoría de cada cambio.

● SANDBOX

Empresa ficticia para practicar. Nunca toca la empresa real.

05 · Referencia técnica

Documentación de sistema · Empresa Operativa Chile

Documento 05 de la serie

Versión del manifiesto 1.5.0 · commit 25f1a5e

Análisis del 2026-09-22

Documento técnico generado desde docs/system-documentation/. La fuente es el Markdown del repositorio: si este PDF y el Markdown difieren, manda el Markdown. Esta aplicación no presenta ni paga nada ante el SII y no es asesoría tributaria.

05 · Referencia técnica

Catálogo de consulta. Para cada función relevante: **firma, propósito, parámetros, retorno, excepciones, efectos secundarios, quién la llama, a quién llama y riesgo al modificarla.**

El riesgo se gradúa así:

Riesgo	Significado
 Crítico	Un error aquí produce una cifra plausible y equivocada, o pierde datos, sin señal visible
 Alto	Rompe una garantía del producto (inmutabilidad, aislamiento, evidencia)
 Medio	Rompe una vista o un flujo, con error visible
 Bajo	Presentación o utilidad; el fallo se ve de inmediato

1 · `packages/chile-tax-rules/index.mjs`

`loadRules(year = 2026)`

Campo	Valor	
Propósito	Devolver el objeto completo de reglas de un año comercial	
Parámetros	<code>`year: number`</code>	<code>string` — año comercial</code>
Retorno	El objeto tal cual está en <code>rules/<año>.json</code> , embebido en <code>rules.generated.mjs</code>	
Excepciones	<code>Error("No hay reglas verificadas para el año comercial <año>. Disponibles: ...")</code>	
Efectos secundarios	Ninguno. Función pura sobre un objeto congelado en el módulo	
La llaman	<code>accounting-engine/index.mjs</code> , <code>tax-equity.mjs</code> , <code>municipal-patent.mjs</code> , <code>workspace.mjs</code> , <code>validate-rules.mjs</code> , la CLI, las vistas <code>empresa</code> y <code>academia</code>	
Llama a	Nada	
Riesgo al modificar	 Crítico. Es la única puerta a las tasas. Hacer que degrade a otro año cuando falta el pedido es explícitamente el peor fallo posible del sistema, y <code>CONTRIBUTING.md</code> lo prohíbe por escrito. Una prueba (<code>tests/rules.test.mjs</code>) verifica que <code>loadRules(1999)</code> lanza	

availableYears()

Devuelve una **copia** de `AVAILABLE_YEARS` (hoy `[2026]`). El `[...AVAILABLE_YEARS]` no es decorativo: impide que un consumidor mute el arreglo del módulo.  **Bajo.**

ruleProvenance(year, ruleName)

Devuelve `{ rule, year, source, lastVerified, note }`. Lanza `Error("Regla desconocida: ...")` si el nombre no existe o no es un objeto. La usa `tests/rules.test.mjs` para exigir que las siete reglas trazables (`iva`, `honorarios`, `ppmProPyme`, `idpcProPyme`, `utm`, `municipalPatent`, `f29`) citen fuente y fecha. 
Medio — si deja de lanzar, la prueba de trazabilidad se vuelve inútil.

2 · packages/accounting-engine/index.mjs

clp(n)

`Math.round(Number(n || 0))`. El peso chileno no usa decimales, y **todo** monto del sistema pasa por aquí. 

Alto: cambiar el redondeo cambia todas las cifras del producto a la vez.

saleFromNet(net, year = 2026)

Campo	Valor
Retorno	<code>{ net, iva, total, rate }</code>
Excepciones	<code>Error("net debe ser un número >= 0")</code> vía <code>assertNonNegative</code>
Efectos	Ninguno
La llaman	<code>f29Basic</code> , <code>journal</code> , CLI (<code>venta</code>), pruebas
Llama a	<code>loadRules</code> , <code>clp</code>
Riesgo	 Alto — el IVA de todas las ventas derivadas sale de aquí

purchaseFromNet(net, year = 2026)

Igual que la anterior más `{ kind: 'purchase', note }`. La `note` es contenido, no adorno: recuerda que el IVA sólo es crédito fiscal si cumple los requisitos legales y está bien respaldado.  **Alto.**

honorariumFromGross(gross, year = 2026)

Retorna `{ gross, retention, liquid, rate }` aplicando `r.honorarios.retentionRate` (15,25 % en 2026).  **Alto** — la retención es dinero que la empresa debe enterar.

ppmFromSalesNet(salesNet, year = 2026, rate)

`rate` opcional; si falta, usa `r.ppmProPyme.initialYearRate`. Retorna `{ base, rate, ppm }`.  **Medio.**

f29Basic(input, year = 2026) **Crítico**

La función más consultada del motor.

```
f29Basic({
  salesNet = 0, purchasesNet = 0, previousVatCredit = 0, honorariaGross = 0,
  debitVat, creditVat, ppmRate
}, year = 2026)
```

Campo	Valor
Propósito	Producir el borrador de control del F29 de un período
Dos modos	Derivado: sólo netos → el IVA se calcula con la tasa general. Documentos: si llega <code>debitVat</code> o <code>creditVat</code> , se usan los montos reales
Retorno	<code>{ origin, debitVat, currentCreditVat, previousVatCredit, availableCreditVat, vatPayable, nextVatCredit, ppm, honorariaWithholding, estimatedF29Payment, limitations[] }</code>
Excepciones	<code>assertNonNegative</code> sobre los seis montos
Efectos	Ninguno
La llaman	Vistas <code>panel</code> e <code>impuestos</code> , <code>simulateScenario</code> , CLI (<code>f29</code> , <code>resumen</code>)
Llama a	<code>saleFromNet</code> , <code>purchaseFromNet</code> , <code>ppmFromSalesNet</code> , <code>honorariumFromGross</code>

Lo no obvio, y es lo importante: `origin` distingue los dos modos, y el arreglo `limitations`

crece en modo derivado con un aviso extra. La diferencia no es cosmética: una factura exenta, una nota de crédito o un redondeo del emisor hacen que el IVA real no sea $\text{neto} \times 19\%$. Si el motor recalculara sobre los netos cuando ya tiene los montos declarados, le cobraría al usuario un IVA que legalmente no debe.

`vatPayable = max(0, debit - (credit + previousVatCredit))` y `nextVatCredit = max(0, (credit + previousVatCredit) - debit)`. Las dos ramas del mismo `max`, y por eso nunca pueden ser ambas positivas.

Riesgo: ■ **Crítico.** Es la cifra que el usuario compara con la propuesta del SII. Cualquier cambio necesita prueba en `tests/f29.test.mjs`.

`f29DueDates(period, year = 2026)`

Campo	Valor
Parámetros	<code>period: 'YYYY-MM'</code>
Retorno	<code>{ period, general, internetWithPayment, internetWithoutPayment, checkHolidays: true, source }</code> — cada fecha es <code>{ date, shiftedFromWeekend }</code>
Excepciones	<code>Error("period debe tener formato YYYY-MM")</code>
La llaman	Vistas <code>panel</code> y <code>obligaciones</code> , CLI (<code>vencimientos</code>)

Lo no obvio: devuelve **tres** fechas y no una, porque el plazo del SII depende de si se presenta por internet y de si el formulario resulta con pago. Y `checkHolidays: true` viaja siempre: el cálculo desplaza sábados y

domingos pero **no conoce los feriados legales chilenos**. Ese campo es la declaración honesta de la limitación, no una bandera de configuración.

Riesgo: ■ **Alto** — una fecha mal calculada produce una multa.

`municipalPatent({ capital, rate, utm }, year = 2026)`

Estimación **rápida** sobre una cifra de capital **dada**. Aplica la tasa, respeta el mínimo (1 UTM) y el máximo (8.000 UTM), y lanza si la tasa está fuera del rango legal. **No decide qué capital corresponde usar**: para eso está `calculateMunicipalPatent`. Se conserva porque la CLI y los laboratorios la usan para enseñar el tope mínimo. ■ **Medio**.

`idpcProPyme({ incomeReceived, expensesPaid, adjustments }, year = 2026)`

`base = max(0, ingresos - gastos + ajustes)`, `estimatedIdpc = base × rate`. ■ **Medio** — declarado como referencia educativa en la propia regla.

`journal(operation, year = 2026)` ■ **Alto**

Devuelve el arreglo de asientos explicados de una operación. Soporta **nueve** tipos y **lanza** `Error("Tipo no soportado: ...")` para cualquier otro.

type	Asiento resultante
<code>capital_contribution</code>	Debe Banco / Haber Capital
<code>shareholder_loan</code>	Debe Banco / Haber Cuenta por pagar al accionista
<code>shareholder_loan_repayment</code>	Debe Cuenta por pagar / Haber Banco
<code>sale</code>	Dos líneas: ingreso neto + IVA débito
<code>purchase</code> / <code>expense</code>	Dos líneas: neto + IVA crédito
<code>honorarium</code>	Dos líneas: líquido pagado + retención por enterar
<code>owner_withdrawal</code>	Debe Cuenta accionista / retiros — no es gasto
<code>tax_payment</code>	Debe Impuestos por pagar / Haber Banco

Lo no obvio: que `capital_contribution` y `shareholder_loan` sean tipos distintos es una decisión de producto, no de implementación. Los dos entran por el mismo banco y son opuestos contablemente —uno acredita patrimonio, el otro pasivo exigible— y por eso la aplicación **obliga a elegir la naturaleza del depósito** en vez de suponer que todo lo que pone el dueño es capital.

`simulateScenario(scenario, year = 2026)`

Recorre `scenario.operations`, acumula totales por tipo, genera los asientos y devuelve `{ profile, totals, f29, entries }`. Sólo la usan la CLI (`escenario`) y las pruebas.

Lo no obvio: `totals.capital` y `totals.shareholderLoans` se cuentan por separado **a propósito** — son dos entradas de dinero del mismo dueño con efectos patrimoniales opuestos. ■ **Medio**.

3 · `packages/accounting-engine/tax-equity.mjs`

`allowsSimplifiedTaxEquity(taxRegime)` ■ Crítico

Devuelve `true` sólo si el régimen coincide con `/pro\s*pyme\s*general/i` o `/14\s*D\s*N?\.\?\s*3/i`, y `false` de forma explícita si el texto contiene «transparente». Ese corte previo importa: sin él, «Pro Pyme Transparente» pasaría el primer patrón.

Riesgo: ■ Crítico. Aplicar la fórmula simplificada a quien no califica da una cifra plausible y equivocada, que es el peor resultado posible. `tests/tax-equity.test.mjs` lo cubre.

`calculateTaxEquity(input)` ■ Crítico

```
calculateTaxEquity({
  fiscalYear, taxRegime = '', assets, liabilities, nonEffectiveValues = 0,
  equityMovements = {}, taxAdjustments = {}, openingTaxEquity = null,
  operations = {}, method, rulesYear
})
```

Campo	Valor
Retorno	Objeto con <code>calculatedCPT</code> , <code>breakdown[]</code> , <code>calculationMethod</code> , <code>formula</code> , <code>legalBasis</code> , <code>rulesYear</code> , <code>status: 'ESTIMADO'</code> , <code>assumptions[]</code> , <code>warnings[]</code> , <code>evidence[]</code>
Excepciones	Falta <code>fiscalYear</code> entero · método desconocido · el año sin <code>taxEquity</code> en sus reglas
Efectos	Ninguno
La llaman	<code>workspace.taxEquityFor()</code> , CLI (<code>cpt</code>)
Llama a	<code>loadRules</code> , <code>allowsSimplifiedTaxEquity</code> , y las privadas <code>general()</code> / <code>simplified()</code>

Las dos reglas de diseño, citadas del propio módulo: (1) *nunca devuelve sólo un número* —un CPT sin explicación no sirve ni para declarar ni para discutirlo con un contador—; (2) *nunca aplica la fórmula simplificada a quien no califica*, y si alguien la fuerza con `method`, añade una advertencia explícita en `warnings`.

Riesgo: ■ Crítico. El CPT determina la base de la patente municipal del año siguiente, y esa cadena es la lección central del producto.

`general({...})` — privada, método art. 41 N.º 1

`CPT = activos - valoresSinInversiónEfectiva - pasivosExigibles + ajustesPositivos - ajustesNegativos`

Si faltan `assets` o `liabilities`, **no falla**: calcula y añade a `warnings` que la cifra es referencial, y marca `complete: false`. ■ Crítico.

`simplified({...})` — privada, método art. 14 D) N.º 3 (j)

```
bruto = capitalAportado + basesImponibles + participaciones + ajustesPositivos  
      - disminuciones - pérdidas - art21Pagado - retiros - ajustesNegativos  
calculado = max(0, bruto)
```

Lo no obvio y jurídicamente relevante: cuando `bruto < 0`, la letra (j) manda considerar \$0. El módulo lo aplica, expone `rawCPT` con el negativo real, marca `flooredAtZero: true` y añade un `warning` con la cifra bruta. Perder cualquiera de esas tres señales convertiría un dato en una afirmación falsa. ■ **Crítico.**

4 · packages/accounting-engine/municipal-patent.mjs

calculateMunicipalPatent(input) Crítico

Campo	Valor	
Parámetro decisivo	`businessStage: 'NEW_BUSINESS' \	'ESTABLISHED_BUSINESS'
Retorno	{ period, businessStage, baseCapital, baseOrigin, rate, rateStatus, rateSource, legalRateRange, rawPatent, minimumPatent, maximumPatent, annualPatent, semesterAmount, cappedBy, utm, utmPeriod, municipality, legalBasis, source, rulesYear, status, breakdown[], assumptions[], warnings[] }	
Excepciones	businessStage inválido · year no entero · tasa no numérica · tasa fuera del rango legal · UTM ≤ 0 · el año no trae ninguna UTM verificada	
La llaman	workspace.municipalPatentFor(), vista capital, CLI (patente-municipal)	
Llama a	loadRules, resolveRate, resolveUtm	

La bifurcación del art. 24, que es la razón de existir del módulo:

```
NEW_BUSINESS      → base = initialOwnCapital (capital propio inicial DECLARADO)
ESTABLISHED_BUSINESS → base = taxEquity (CPT del balance al 31-12 anterior)
```

Después: `base - deducciones`, con piso en cero; y si hay `allocatedCapital` (prorrateso entre sucursales del art. 25), ése **sustituye** el resultado anterior. Luego `annualPatent = min(máximo, max(mínimo, base × tasa))`, con `cappedBy` indicando cuál tope actuó.

Riesgo:  **Crítico.** Antes de este módulo la patente se estimaba con `capital enterado × tasa mínima` siempre, para cualquier año; eso sólo se parece a la ley el primer ejercicio, y ni siquiera del todo.

resolveRate({...}) — privada Alto

Precedencia: `municipalRate` explícita → tasa de una municipalidad con `status: 'VERIFIED'` → **el mínimo legal como supuesto declarado.**

Lo no obvio: una tasa sólo cuenta como verificada si viene de la municipalidad **y** no fue sobrescrita a mano. Incluso una tasa explícita dentro del rango legal genera `UNVERIFIED_RATE_WARNING`. El sistema prefiere sobreavisar a dar por acreditado algo que no lo está.

resolveUtm({...}) — privada Alto

La UTM cambia todos los meses y los topes de la patente están en UTM, así que arrastrar la de otro período cambia la cifra **en silencio**. La función siempre declara cuál usó, y añade un `warning` distinto según el caso:

pidieron un mes que no existe, o el mes elegido no pertenece al período.

5 · packages/company-operations/workspace.mjs — clase

CompanyWorkspace

`constructor({ store, mode = 'real' })` . Lanza si `mode` no es `real` ni `sandbox` , o si falta `store` . **No crea el almacén**: lo recibe. Ésa es la razón de que el mismo motor corra en cuatro sitios.

Auditoría

Método	Retorno	Efectos	Riesgo
<code>audit(action, detail)</code>	La fila creada	<code>store.append(KEY.audit, {id, at, mode, action, detail})</code>	Alto — la llaman los 17 puntos de mutación
<code>listAudit()</code>	Todas las filas	Ninguno	Bajo

Lo no obvio: no existe ninguna operación de borrado sobre `KEY.audit` , y es deliberado. Si un registro pudiera borrarse sin dejar huella, la bitácora no serviría como evidencia de nada.

Ficha, capital y municipalidad

Método	Propósito	Excepciones	Riesgo
<code>getCompany()</code> / <code>saveCompany(c)</code>	Ficha de empresa; añade <code>updatedAt</code> , <code>createdAt</code> y <code>mode</code>	—	 Medio
<code>getCapitalProfile()</code>	Ficha de capital normalizada, migrando el modelo antiguo en lectura	—	 Alto
<code>saveCapitalProfile(p)</code>	Valida y guarda; sincroniza <code>company.capital</code> con <code>capitalEnterado</code>	Los <code>errors</code> de <code>validateCapitalProfile</code> unidos con espacio	 Alto
<code>listEquityMovements()</code>	El ledger explícito	—	 Bajo
<code>addEquityMovement(m, {registerCashMovement})</code>	Añade y ordena por fecha; opcionalmente crea la operación de caja enlazada	Las de <code>normalizeEquityMovement</code>	 Crítico
<code>deleteEquityMovement(id)</code>	<code>boolean</code>	—	 Alto
<code>effectiveEquityMovements()</code>	Une el ledger explícito con las operaciones de caja antiguas no enlazadas	—	 Crítico
<code>capitalPosition({until})</code>	Las cuatro cifras de capital por separado, nunca fundidas	—	 Crítico
<code>getMunicipalProfile()</code> / <code>saveMunicipalProfile(p)</code>	Ficha municipal normalizada	—	 Medio

El riesgo crítico de `effectiveEquityMovements()`, explicado: une dos orígenes sin duplicar. El ledger explícito manda; las operaciones de caja antiguas se incorporan como movimientos derivados

salvo las que ya están enlazadas por `equityMovementId`. Romper ese filtro contaría cada aporte dos veces al derivar el capital enterado, y el capital enterado alimenta el CPT, que alimenta la patente.

El riesgo crítico de `capitalPosition()`: el capital enterado real es el **mayor** entre lo declarado en la ficha y lo que suman los movimientos. Quedarse con el declarado subestimaría la patente cuando el usuario registró aportes posteriores.

Ejercicio anual

Método	Propósito	Excepciones	Riesgo
<code>periodsOfYear(year)</code>	Períodos <code>YYYY-MM</code> con movimiento de ese año	—	 Bajo
<code>yearSummary(year)</code>	Agregado del año, sumando sus períodos	—	 Alto
<code>estimatedBalance(year)</code>	Balance estimado al cierre, con <code>estimated: true</code> y <code>limitations[]</code>	—	 Alto
<code>taxEquityFor(year, overrides)</code>	CPT del año; <code>overrides</code> reemplaza lo propuesto por lo declarado	Las de <code>calculateTaxEquity</code>	 Crítico
<code>businessStageFor(year)</code>	<code>NEW_BUSINESS</code> o <code>ESTABLISHED_BUSINESS</code>	—	 Crítico
<code>municipalPatentFor(year, overrides)</code>	Patente del período con toda su trazabilidad	Las de <code>calculateMunicipalPatent</code>	 Crítico
<code>listAnnualCloses()</code> / <code>getAnnualClose(year)</code>	Cierres guardados	—	 Bajo
<code>closeFiscalYear(year, {...})</code>	Fotografía inmutable del ejercicio	Año no entero · ejercicio ya cerrado	 Crítico
<code>reopenFiscalYear(year, reason)</code>	Elimina el cierre dejando rastro	Ejercicio no cerrado · motivo vacío	 Alto
<code>capitalHistory()</code>	Historial por año, sin recalcularlo cerrado	—	 Alto

`businessStageFor(year)` , con el detalle que decide la cifra:

```
Number(String(start).slice(0, 4)) >= Number(year) ? 'NEW_BUSINESS' : 'ESTABLISHED_BUSINESS'
```

donde `start` es `fechaInicioActividades || fechaConstitucion` . Sin ninguna de las dos, devuelve `NEW_BUSINESS` . Es empresa nueva **en el año en que inicia actividades**; desde el siguiente, en funcionamiento.

`closeFiscalYear()` — **lo no obvio y valioso**: el snapshot guarda `legalRulesVersion` , una copia de la **versión de las reglas** con que se calculó, no una referencia al archivo. Dentro de tres años `rules/2026.json` puede haberse corregido, y el cierre tiene que seguir explicando con qué normas se calculó ese día. El objeto se devuelve con `Object.freeze()` , y el cierre incluye `municipalPatentBaseForNextPeriod` , que es lo que conecta un ejercicio con el siguiente.

`capitalHistory()` — **lo no obvio**: si falta el archivo de reglas del año pedido, no se rinde. Vuelve a intentar con el último año verificado **anterior** y marca el resultado con `simulatedWithRulesYear` . La distinción es exacta: enseñar el mecanismo sí, fingir que la cifra es la del período no.

Constitución

Método	Propósito	Excepciones	Riesgo
<code>listFormationSteps()</code>	Los 9 pasos del catálogo, fusionados con lo guardado	—	 Medio
<code>updateFormationStep(step)</code>	Actualiza uno	Paso desconocido · <code>status</code> inválido · <code>done</code> sin <code>evidenceRef</code>	 Alto
<code>formationProgress()</code>	<code>{ total, done, percent, ready }</code>	—	 Bajo

Lo no obvio en `listFormationSteps()` : el catálogo vive en el **código**, no en los datos. Cuando se agrega un trámite nuevo, las empresas ya creadas lo ven aparecer en vez de quedarse con una lista congelada del día en que se instalaron. Una prueba lo verifica.

Operaciones

Método	Excepciones	Riesgo
<code>listTransactions()</code>	—	 Bajo
<code>addTransaction(tx)</code>	<code>kind</code> no soportado · <code>date</code> mal formada · período cerrado	 Alto
<code>updateTransaction(id, patch)</code>	No encontrada · período de origen cerrado · período de destino cerrado	 Alto
<code>deleteTransaction(id)</code>	Período cerrado	 Alto

Lo no obvio en `updateTransaction` : comprueba el cierre **dos veces** — el período de la operación original y el de la operación resultante. Sin la segunda, se podría *mover* una operación a un mes ya cerrado, que es la misma violación por la puerta de atrás.

Campos con valor por defecto que conviene conocer: `deductible: tx.deductible !== false` y `vatCreditEligible: tx.vatCreditEligible !== false` . Es decir, **por omisión todo es deducible y toda da crédito**; hay que marcar explícitamente lo contrario.

Obligaciones y períodos

Método	Excepciones	Riesgo
<code>upsertObligation(ob)</code>	Sin <code>type</code> · <code>done</code> sin <code>evidenceRef</code>	 Alto
<code>deleteObligation(id)</code>	—	 Medio
<code>listPeriods()</code>	—	 Bajo
<code>periodSummary(period)</code>	—	 Alto
<code>vatCarryForwardInto(period)</code>	—	 Crítico
<code>isPeriodClosed(period)</code> / <code>listClosedPeriods()</code> / <code>listPeriodCloses()</code>	—	 Bajo
<code>closePeriod(period, checklist)</code>	Formato inválido · ya cerrado	 Alto
<code>reopenPeriod(period, reason)</code>	No cerrado · motivo vacío	 Alto

`periodSummary()` — **lo no obvio**: el crédito fiscal suma **sólo** las filas con `vatCreditEligible`; las demás se acumulan aparte en `rejectedVat`. Y `deductibleExpenses` suma

sólo las filas con `deductible`. Son dos casillas independientes, y esa independencia es exactamente la lección de la Academia: un almuerzo con cliente sale plata igual, pero su IVA no es recuperable y su neto no es gasto deducible.

`vatCarryForwardInto(period)` —  **Crítico y el más fácil de romper**:

```
let carry = 0;
for (const p of this.listPeriods()) {
  if (p >= period) break;
  const s = this.periodSummary(p);
  carry = Math.max(0, (s.creditVat + carry) - s.debitVat);
}
```

Recorre **toda** la historia desde el principio en cada llamada. Sin este arrastre la app le cobraría al usuario un IVA que legalmente no debe. **No aplica reajuste** (art. 27 del D.L. 825), y eso está declarado como limitación en el código y en `ARCHITECTURE.md`. Coste: es $O(n^2)$ sobre el número de períodos — irrelevante con decenas de meses, comentado en [15 · Riesgos](#).

Respaldo y salud

Método	Retorno	Excepciones	Riesgo
<code>exportAll()</code>	Objeto plano con <code>format: 'empresa-operativa-chile/backup'</code> y <code>formatVersion: 2</code>	—	 Alto
<code>importAll(payload, {replace})</code>	<code>{ imported, replace, transactions }</code>	Formato ajeno · <code>formatVersion</code> fuera de <code>[1, 2]</code>	 Crítico
<code>backup()</code>	<code>{ name, location }</code>	—	 Medio
<code>listBackups()</code>	Nombres	—	 Bajo
<code>healthCheck(period)</code>	<code>{ period, level, issues[], formation, summary }</code>	—	 Medio

`importAll()` — **dos cosas no obvias**. Primera: acepta `formatVersion 1 y 2`. Los respaldos v1 se importan igual; los campos nuevos simplemente no vienen y quedan vacíos. Romper los respaldos que la gente ya tiene guardados sería peor que cualquier ventaja de limpiar el formato. Segunda: en modo

fusión, un ejercicio ya cerrado **no se pisa** desde un respaldo. Si se pudiera, el cierre dejaría de ser inmutable por la puerta de atrás. Hay una prueba dedicada a eso.

`healthCheck()` — **la decisión de producto**: deliberadamente **no** dice «todo en orden» cuando faltan evidencias. Emite nueve familias de observación (`company.missing` , `company.rut` , `formation.pending` , `evidence.missing` , `vat.rejected` , `obligations.overdue` , `capital.incomplete` , `capital.pending` , `municipal.missing` / `municipal.rate` , `annual-close.missing`) y el `level` resultante es el peor de todos.

`seedSandboxWorkspace(ws)` — función exportada aparte

Campo	Valor
Excepciones	<code>Error("El sandbox sólo puede sembrarse en modo sandbox")</code>
Idempotencia	Si ya hay transacciones, devuelve <code>ws</code> sin tocar nada
Riesgo	 Alto — sembrar sobre EMPRESA REAL sería una contaminación de datos irreversible. Hay una prueba dedicada

Siembra un ejercicio completo de julio a diciembre de 2026: capital social \$3.000.000 con sólo \$1.000.000 enterado al partir, un notebook aportado en especie, un préstamo del accionista, un retiro y 18 operaciones. Las cifras son distintas **a propósito**: es la situación real más frecuente y la que el modelo anterior, con un solo campo `capital` , no podía representar.

6 • packages/company-operations/capital.mjs

Función	Propósito	Excepciones	Riesgo
<code>equityMovementKind(id)</code>	Metadatos de uno de los 9 tipos, o <code>null</code>	—	 Bajo
<code>normalizeCapitalProfile(company)</code>	Normaliza y migra el modelo antiguo de un solo campo <code>capital</code>	—	 Crítico
<code>capitalPendingToPay(profile)</code>	<code>max(0, suscrito - enterado)</code> o <code>null</code>	—	 Alto
<code>validateCapitalProfile(profile)</code>	<code>{ valid, errors[], warnings[] }</code>	—	 Alto
<code>normalizeEquityMovement(movement)</code>	Valida y normaliza antes de guardar	Tipo no soportado · sin fecha <code>YYYY-MM-DD</code> · monto ≤ 0 · aporte en bienes sin descripción o sin aportante	 Alto
<code>summarizeEquityMovements(movements, {until})</code>	Agrega hasta una fecha de corte	—	 Crítico

`normalizeCapitalProfile()` — la migración que no puede corromper nada. Si la ficha guardada sólo tenía `capital`, esa cifra pasa a `capitalEnterado` —que es lo que el campo rotulaba— y `capitalSocial` y `capitalSuscrito` quedan marcados en `pendingConfirmation`. **No se inventan.** Y la migración ocurre **en lectura**: no toca el almacén, así que si nadie vuelve a guardar, el dato original sigue intacto en disco. Instalar esta versión no puede corromper datos existentes.

`capitalPendingToPay()` devuelve `null` y no cero cuando el capital suscrito no se conoce. Cero significaría «no falta nada», y eso es una afirmación que aquí nadie puede hacer.  **Alto**: quien consuma esta función tiene que distinguir `null` de `0`.

`summarizeEquityMovements()` — lo no obvio: `capitalEnteradoPorMovimientos` sólo suma los tipos con `entersCapital: true` (4 de los 9). Un préstamo del accionista entra por el mismo banco y **no suma un peso** ahí.

7 · packages/company-operations/store.mjs y node-store.mjs

createMemoryStore() Bajo

Todo pasa por `JSON.parse(JSON.stringify(...))`. El clonado no es paranoia: sin él, un test que mutara el objeto devuelto contaminaría el almacén y el siguiente test.

createWebStore({ namespace, storage = globalThis.localStorage }) Alto

Lanza si falta `storage` o `namespace`. Toda clave se prefija `${namespace}:${key}`. Ese prefijo es la **separación EMPRESA REAL / SANDBOX**: no una bandera, un espacio de nombres. Una prueba verifica el aislamiento sobre el mismo origen.

`readRaw` absorbe el error de `JSON.parse` y devuelve el `fallback`. **INFERENCIA**: un valor corrupto en `localStorage` se lee como «vacío» en vez de romper la app — bueno para la robustez, pero también significa que una corrupción **no avisa**. Registrado en [15 · Riesgos](#).

createNodeStore({ rootDir, mode }) Alto

Aspecto	Detalle
Directorio	<code>path.join(path.resolve(rootDir), mode)</code> — la separación por modo es física
Subdirectorios	Crea <code>evidence/</code> y <code>backups/</code> al construirse
<code>write</code>	<code>writeJsonAtomic</code> : escribe a <code>\${file}.\${pid}.tmp</code> y hace <code>rename</code> . Un corte de luz deja intacto el archivo anterior
<code>append</code>	<code>fs.appendFileSync</code> de una línea NDJSON. Nunca reescribe el archivo completo
<code>readAll</code>	Si una línea no parsea, devuelve <code>{ corrupt: true, line }</code> en vez de perderla
<code>saveSnapshot</code>	Copia <code>snapshot.json</code> más los siete archivos individuales

Lo no obvio: **FILES** mapea siete claves a nombres de archivo; las demás (`equity-movements`, `annual-closes`, `municipal-profile`) caen al `fallback ${key}.json`. Consecuencia real: `saveSnapshot` copia sólo las siete de **FILES**, así que un respaldo de archivos individuales **no incluye** los movimientos patrimoniales ni los cierres anuales — aunque `snapshot.json`, que se escribe desde `exportAll()`, sí los lleva completos. Registrado en [15 · Riesgos](#).

8 · packages/company-operations/rut.mjs

Función	Retorno	Excepciones	Riesgo
<code>cleanRut(input)</code>	Texto sin puntos ni espacios, en mayúsculas	—	 Bajo
<code>rutCheckDigit(body)</code>	<code>'0' ... '9'</code> o <code>'K'</code>	<code>Error("El cuerpo del RUT debe tener dígitos")</code>	 Alto
<code>validateRut(input)</code>	<code>{valid: true, formatted, body, dv}</code> o <code>{valid: false, reason}</code>	—	 Alto

Módulo 11 con factor cíclico 2→7. `rest === 11 → '0'`, `rest === 10 → 'K'`. Acepta cuerpos de 7 u 8 dígitos con o sin guion. **Riesgo:** un RUT mal validado no falla el día que se escribe; se propaga a documentos y conciliaciones y reaparece meses después como un descuadre cuyo origen ya nadie recuerda.

9 • packages/shortcuts/index.mjs

`resolveShortcut(event, { typing = false })`  **Medio**

Devuelve el atajo coincidente o `null`. Trata `ctrlKey` y `metaKey` como el mismo modificador (compatibilidad con macOS). El caso `digits` resuelve `Alt + 1...9` devolviendo `{...s, index}`.

La regla que evita el problema clásico de los atajos: cuando `typing` es `true`, sólo pasan los que llevan `alt` o `ctrl`, y `Escape`, que es la salida de emergencia de cualquier diálogo. Escribir «no» en una descripción de gasto no abre nada. Hay una prueba dedicada.

`shortcutsOf`, `shortcutsByGroup`, `keysFor` —  **Bajo.**

10 · `packages/glossary/` y `packages/onboarding/`

Función	Propósito	Riesgo
<code>term(id)</code>	Un término o <code>null</code>	 Bajo
<code>searchTerms(query)</code>	Búsqueda sin acentos ni mayúsculas sobre término, resumen, definición e id	 Bajo
<code>termsByCategory()</code>	Agrupados en el orden de <code>CATEGORIES</code> , saltando las vacías	 Bajo
<code>glossary.danglingReferences()</code>	Ids de <code>related</code> / <code>notToConfuseWith</code> que no existen	 Medio
<code>stage(id)</code> / <code>stagesByPhase()</code>	Etapas de la ruta	 Bajo
<code>onboarding.danglingReferences({views, glossaryIds, formationSteps})</code>	Referencias rotas a fases, vistas, trámites o términos	 Medio

Las dos funciones `danglingReferences()` son la razón de que el glosario y la guía no puedan quedar con enlaces rotos: `build-glossary.mjs` y `build-guide.mjs` **abortan** si devuelven algo, y las pruebas las ejecutan también.  **Medio** — quitarlas no rompe nada visible hoy y deja entrar enlaces rotos mañana.

11 · Interfaz — apps/web/src/lib/

dom.js

Función	Riesgo	Nota
<code>html</code> (plantilla etiquetada)	 Crítico	Escapa toda interpolación por defecto. Los datos los escribe el usuario —descripciones, RUT, motivos de reapertura— y se muestran en tablas y en la bitácora
<code>raw(value)</code>	 Crítico	La única puerta para insertar HTML sin escapar. Cada <code>raw()</code> es una decisión consciente
<code>esc(value)</code>	 Alto	Escapa <code>& < > " ' </code>
<code>attempt(fn, okMessage)</code>	 Medio	Traduce cualquier excepción a un <code>toast</code> . El mensaje del motor es la explicación de la regla
<code>openModal({...})</code>	 Medio	Cierra con Escape y con el fondo; devuelve el <code><form></code> como <code>FormData</code>
<code>confirmAction({...})</code>	 Medio	Confirmación explícita para lo irreversible
<code>fmtCLP</code> , <code>fmtNumber</code> , <code>fmtPercent</code> , <code>fmtPeriod</code> , <code>fmtDate</code> , <code>fmtDateTime</code>	 Bajo	<code>fmtDate</code> parte la cadena ISO en vez de construir un <code>Date</code> , para no depender de la zona horaria del equipo

state.js

Función	Riesgo	Nota
<code>ws(mode = state.mode)</code>	 Alto	Devuelve el <code>CompanyWorkspace</code> del modo, creándolo perezosamente. Los dos modos viven a la vez , con almacenes distintos
<code>ensureSandboxSeeded()</code>	 Alto	Sólo siembra el sandbox. La empresa real nunca se siembra
<code>setMode</code> / <code>setTheme</code> / <code>toggleTheme</code>	 Bajo	Persisten en <code>localStorage</code> bajo la clave <code>...:prefs</code>
<code>selectablePeriods()</code>	 Bajo	Períodos con movimiento + el actual + el seleccionado, del más nuevo al más viejo

platform.js

Función	Riesgo	Nota
<code>PLATFORM</code>	 Medio	'windows' si existe <code>__TAURI__.core</code> ; 'android' si <code>Capacitor.isNativePlatform()</code> ; si no, 'web'
<code>createPlatformStore(mode)</code>	 Alto	Envuelve <code>createWebStore</code> y añade el espejo. Best effort : si el disco falla, la app sigue con <code>localStorage</code> y avisa por consola
<code>hydrateFromDisk()</code>	 Crítico	Sólo repone lo que falta : si la WebView ya tiene el dato, gana el dato vivo. Sin esa condición, un espejo viejo pisaría una sesión en curso
<code>saveTextFile(filename, contents, mime)</code>	 Medio	En Windows llama al comando Rust; en web y Android crea un <code>Blob</code> y dispara la descarga
<code>pickTextFile(accept)</code>	 Bajo	<code><input type="file"></code> efímero
<code>openExternal(url)</code>	 Medio	En Android usa el plugin Browser de Capacitor para no dejar la app atrapada en la WebView

El espejo, en detalle: `mirror()` es un `setTimeout` de **180 ms** que se reinicia en cada escritura (*debounce*). Vuelca **todas** las claves de `localStorage` que empiecen por el *namespace* del modo y las manda como un único JSON a `save_workspace`.

12 · Backend Rust — apps/contador-desktop/src-tauri/src/lib.rs

Función	Firma	Riesgo	Nota
<code>safe_mode(mode)</code>	<code>&str → Result<&'static str, String></code>	 Crítico	Acepta exactamente <code>"real"</code> y <code>"sandbox"</code> . Sin ella, <code>mode</code> acabaría concatenado en una ruta de disco y <code>../algo</code> sería escritura arbitraria. Dos pruebas Rust la cubren
<code>data_dir(app)</code>	<code>→ Result<PathBuf, String></code>	 Medio	<code>app_data_dir()</code> + <code>create_dir_all</code>
<code>workspace_file(app, mode)</code>	<code>→ Result<PathBuf, String></code>	 Alto	<code>data_dir/{safe_mode(mode)}.json</code>
<code>load_workspace(app, mode)</code>	<code>comando · → Result<Option<String>, String></code>	 Medio	<code>None</code> la primera vez
<code>save_workspace(app, mode, payload)</code>	<code>comando · → Result<(), String></code>	 Crítico	Valida que el payload sea JSON antes de tocar el disco , y después escribe a <code>.tmp</code> + <code>rename</code>
<code>export_file(app, filename, contents)</code>	<code>comando · → Result<String, String></code>	 Alto	Toma sólo <code>file_name()</code> del nombre recibido y rechaza vacío o con punto inicial. Escribe en <code>Documentos/Empresa Operativa Chile</code>
<code>app_info(app)</code>	<code>comando · → Result<Value, String></code>	 Bajo	Versión y directorio de datos

Los cuatro comandos están registrados en `invoke_handler`. Las capacidades declaran `permissions: ["core:default"]` y nada más: **el acceso a disco pasa exclusivamente por estos cuatro comandos, que validan sus argumentos.**

13 · Servidor — apps/empresa-operativa/server.mjs

resolveSafe(urlPath) Crítico

```
const full = path.resolve(distDir, `.${path.posix.normalize(rel)}\`);  
if (full !== distDir && !full.startsWith(distDir + path.sep)) return null;
```

Lo no obvio, y es la razón del comentario en el código: la comprobación usa `distDir + path.sep`, no un `startsWith(distDir)` a secas. Sin ese separador, un directorio hermano llamado `dist-privado` pasaría el filtro.

CI comprueba tres vectores de escape (`/../../package.json`, `/%2e%2e%2f...`, `/..%2f...`) y falla si alguno devuelve 200.

Riesgo:  **Crítico.** El servidor sirve datos contables en la máquina de alguien.
